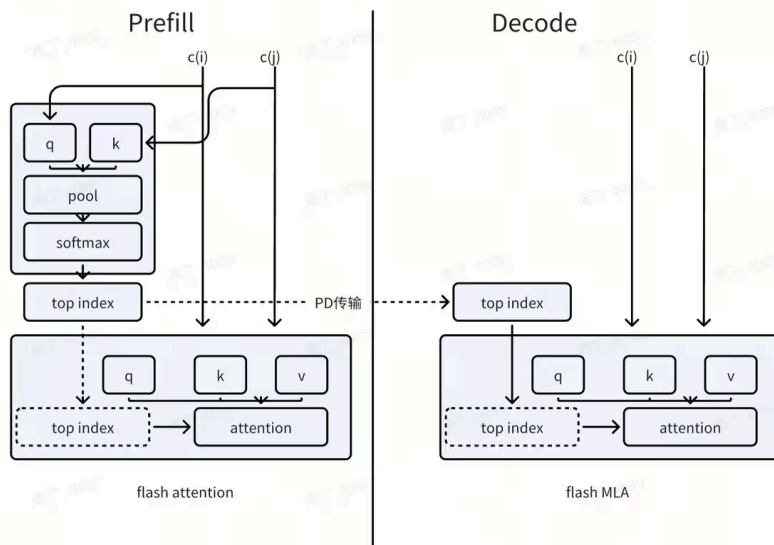




长上下文 (Long context) 的10倍推理加速



Zachary

37 人赞同了该文章

关注

在现代软件开发中，代码项目的复杂性和规模不断增加，传统的短上下文模型已难以满足智能辅助编程的需求。以行业领先的编码产品为例，[GitHub Copilot+](#) 支持约 8k tokens，[Cursor+](#) 默认支持 128k tokens 的上下文窗口，而在 Max Mode 下，支持的上下文窗口可扩展至每个模型的最大值，例如 [Qwen2.5-7B-Instruct-1M+](#) 和 [Qwen2.5-14B-Instruct-1M+](#) 支持高达 1M tokens 的上下文窗口。这一显著提升彰显了长上下文对精准理解和生成代码的关键作用。长上下文使得模型能够一次性捕获跨文件、跨模块的依赖关系和设计意图，提升代码补全、重构、bug 修复等多种任务的效果。

因而长上下文成为 Agent 或者 LLM 未来的主要趋势，但与此同时，带着的一个主要问题就是长上下文的精度与性能问题，我们主要讨论基于长上下文的优化技术，同时会把部分优化后的 kernel 测试结果（对比 flash-attention，在 128k 的长度下，有近乎 10 倍的加速）分享一下。

面临的挑战

Attention distractions⁺带来的长上下文精度问题

在长文本处理场景下，Attention 机制面临着显著的精度挑战，其中 “Attention distractions” 问题尤为突出。虽然全量 Attention (full attention) 付出了平方级别的计算复杂度，但在冗长的上下文中，模型往往难以准确聚焦于真正关键的信息，反而容易被大量无关的内容 “分心”，导致生成结果的质量下降。这种现象被称为 Attention 的 distractions。

其根本原因可能源于 Attention 在实现上的 token-mixing 方式。尽管在长文本中，Attention 矩阵实际上是高度稀疏的（即大部分元素接近零），但这些 near-zero 元素却通常并未被精确设为零。随着上下文长度达到百万级 token 时，这些无效但非零的 Attention 权重在矩阵乘积中不断累积，导致误差逐渐放大，从而影响模型对关键信息的准确定位和整体表现。

关于作者



Zachary

回答

8

文章

8

关注者

584

关注

发私信

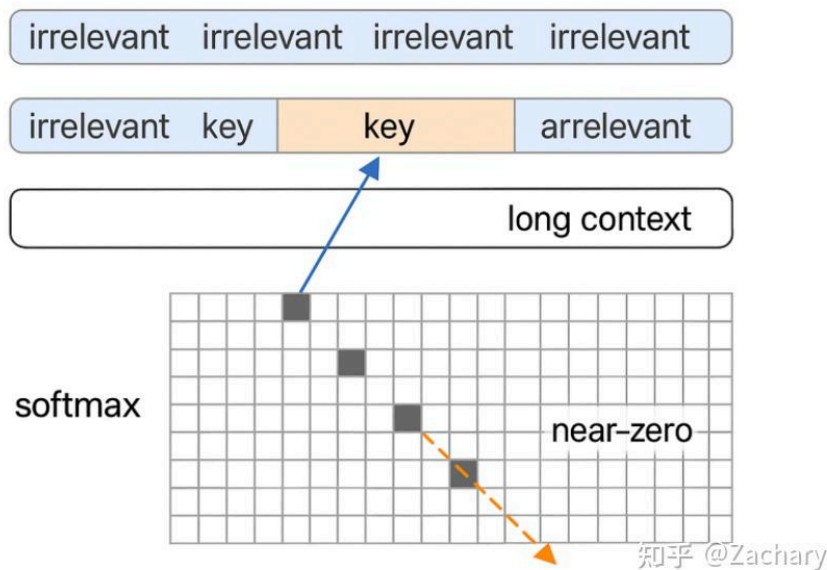
大家都在搜

换一换

- 拼多多被罚 15.2 亿且暴力... 411 万 热
- 爱奇艺再发文回应 AI 艺人... 409 万 新
- 虐死男友 3 岁儿子凶手被判... 408 万 新
- 日本发生 7.5 级地震 407 万 热
- 日本正式允许出口杀伤性... 375 万 新
- 伊朗最新局势 360 万 热
- 女子频繁表白后被确诊「... 344 万 热
- 你在谷歌地球有什么独特... 335 万
- 库克不再担任苹果 CEO 327 万 热
- 沈梦辰连续 4 年做热玛吉脸... 294 万 热



by Irrelevant Context



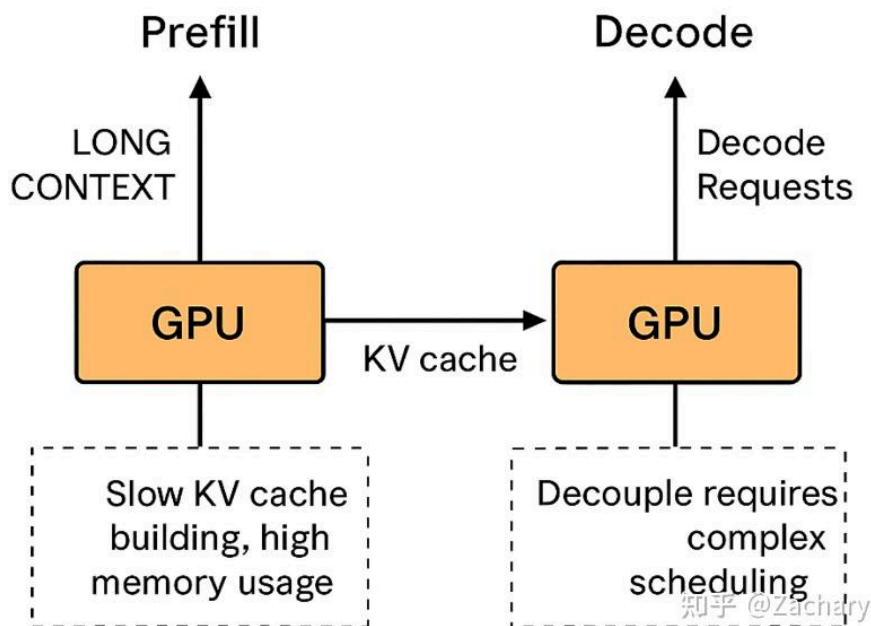
Attention distractions

长上下文带来的性能问题

随着语言模型支持的上下文长度不断扩展到百万级别（例如 Qwen1.5-1M 或 Claude 2.1 等模型），在推理过程中暴露出的性能瓶颈也日益突出，尤其体现在以下几个关键方面：

- **Prefill 阶段计算代价显著上升** 在长上下文推理中，Prefill 阶段 Attention 的计算复杂度则呈 prompt 长度平方增长。当 prompt 长度达到百万 token 时，Prefill 计算的时间和显存占用极其可观，甚至可能超过整个生成阶段的成本，成为推理中的性能瓶颈。
- **KV Cache 管理压力剧增** 长上下文意味着 KV 缓存的尺寸急剧膨胀，这给系统带来了两个维度的压力：
 - 分布式推理中的 KV 同步成本：在 Pipeline/Parallel Decoding (PD) 场景下，KV Cache 需要在多个设备或节点间传输，网络带宽与同步延迟成为限制因素。

for Long Contexts



- 单节点/多线程环境下的内存管理复杂性：管理百万 token 级别的 KV Cache，不仅考验显存容量，还要求高效的内存复用和精细的释放机制，否则很容易造成内存碎片或 OOM。
- 投机采样策略 (Speculative Decoding) 收益降低 在常规短 prompt 场景中，投机采样可通过主-副模型并发生成多个 token 候选，以提高生成速度。然而在长 prompt 情况下，生成过程中 KV Cache 的读写操作频繁、体积巨大，导致副模型的重复预填和比对代价高昂，使得投机采样的性能收益被严重稀释，甚至得不偿失。

当前长上下文加速的主要技术

随着大语言模型在代码生成、多轮对话、文档问答等任务中的深入应用，对长上下文处理能力的需求持续上升。然而，支持百万级 token 的长文本推理，在计算性能与资源管理上带来了严峻挑战，例如高昂的 Prefill 计算成本、复杂的 KV Cache 管理，以及投机采样等传统加速技术的失效。为此，业界和学术界提出了多种优化路径，包括将 Attention 从平方复杂度降为线性的 [Linear Attention⁺](#)，通过 Sparse Attention 结构过滤无关连接减少干扰，以及结合 投机采样优化 和 精细化的推理调度（如 SP、Token-level Pipeline）提升解码效率。这些方法正逐步成为支撑长上下文 LLM 推理的核心技术支柱。本文将围绕上述几个方向，逐一展开详细说明。

Linear Attention

Linear Attention 简介

Linear Attention 是对传统 Softmax Attention 的简化优化，其核心思想是去除归一化操作，采用特征映射将注意力计算转化为线性。特别适合长文本场景，能在保持较好效果的

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- 其中 $Q \in \mathbb{R}^{n \times d}$, $K, V \in \mathbb{R}^{n \times d}$
- 其核心步骤是计算 QK^T ，这是一个 $n \times n$ 的矩阵
- 然后再做 softmax（行方向），得到每个 token 对其它所有 token 的注意力分数
- 最后乘以 V ，输出新的序列向量

! 计算代价为 $O(n^2d)$

知乎 @Zachary

Linear Attention 的目标是将 Attention 的复杂度从 $O(n^2d)$ 降到线性的 $O(nd)$ ，通过重新排列矩阵乘法顺序，如下图所示：

$$\text{LinearAttention}(Q, K, V) = \phi(Q) \cdot (\phi(K)^T V)$$

在工程实现上，因为 Linear Attention 可以将历史的信息累计成一个压缩的矩阵，如下图所示，每次只需更新一次压缩表示（累加矩阵），不需要缓存全部 KV，从而节约大量的内存：

$$\text{context}_t = \sum_{i=1}^t \phi(k_i) v_i^T$$

知乎 @Zachary

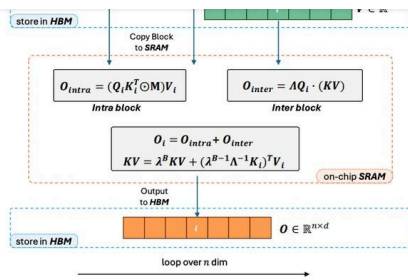
在实际的落地之中，需要考虑 Causal Mask 的影响，即在生成时，不能让当前 token 看到未来的信息，因而 Causal Mask 的时候，并不能直接的进行 KV 计算。

介绍一个 Linear Attention 的算子库 Flash-Linear Attention (github.com/fla-org/flas ...)，Flash-Linear Attention 支持基本所有的 Linear Attention，借鉴了 FlashAttention 的内核融合与内存优化技术，通过将特征映射、累积和归一化等操作融合进单一 GPU kernel，大幅提升执行效率并显著降低内存带宽和显存占用。

Hybrid Lightning Attention

虽说 Linear Attention 的效果比不上 Softmax Attention，但像 MiniMax-01 等模型已然取得不俗的效果，其模型架构为 10* (7 层 Lightning Attention-2 + 1 层 Softmax Attention) 的 80 层结构。RoPE 位置编码仅应用在 Softmax Attention Layer，而且只对每个 Token 一半的维度进行旋转，Hybrid-lightning 相比纯 Lightning Attention 牺牲了一定的速度后，取得了优于纯 Softmax Attention 的效果，特别是在“大海捞针”任务上。随后我们说一下 MiniMax-01 中的 Lightning Attention-2，即通过分块计算 (chunking) 的方法，有效绕过了 Causal Mask 带来的阻碍。

如下图所示，Lightning Attention-2 通过分块的方法，将块的计算分成 intra 和 inter，intra 相当于传统的 mask(QK)V 顺序，计算代价是 n^2 乘以 h ，但是在 inter，因为 h 可以控制块间循环的逻辑，相当于可以忽略 mask 的影响，计算代价是 n 乘以 h^2 。



Initialize mask $M \in \mathbb{R}^{B \times B}$, where $M_{ij} = \lambda^{i-j}$, if $i \geq j$, else 0.
 Initialize $\Lambda = \text{diag}\{\lambda, \lambda^2, \dots, \lambda^B\} \in \mathbb{R}^{B \times B}$.
 Initialize $KV = 0 \in \mathbb{R}^{d \times d}$.
for $1 \leq i \leq T$ **do**
 Load $Q_i, K_i, V_i \in \mathbb{R}^{B \times d}$ from HBM to on-chip SRAM.
 On chip, compute $O_{\text{intra}} = [(Q_i K_i^T) \odot M] V_i$.
 On chip, compute $O_{\text{inter}} = \Lambda Q_i (KV)$.
 On chip, compute $KV = \lambda^B KV + (\lambda^{B-1} \Lambda^{-1} K_i)^T V_i$.
 Write $O_i = O_{\text{intra}} + O_{\text{inter}}$ to HBM as the i -th block of O .
end for
return O .

知乎 @Zachary

Sparse Attention and KV Compression

当前的 Sparse Attention 技术，如 **MoBA⁺** (Mixture of Blockwise Attention) 与 **NSA⁺** (Native Sparse Attention)，正成为解决长上下文计算瓶颈的关键路径。MoBA 借鉴 MoE 思想，将输入序列在上下文维度上细粒度划分，通过块级稀疏计算提升效率，并可与 Full Attention 混合部署，兼顾性能与灵活性；NSA 则从架构上原生支持稀疏模式，如滑动窗口和全局 token 路由，使注意力机制更加高效、可扩展。两者均致力于在不牺牲模型能力的前提下，显著降低推理开销，为百万 token 级的长文本处理提供了重要支撑。

MoBA

MoBA 的目标是在不增加模型参数量的前提下，替代全注意力机制，实现对长序列的高效、低冗余建模，从而提升模型的可扩展性和推理速度。

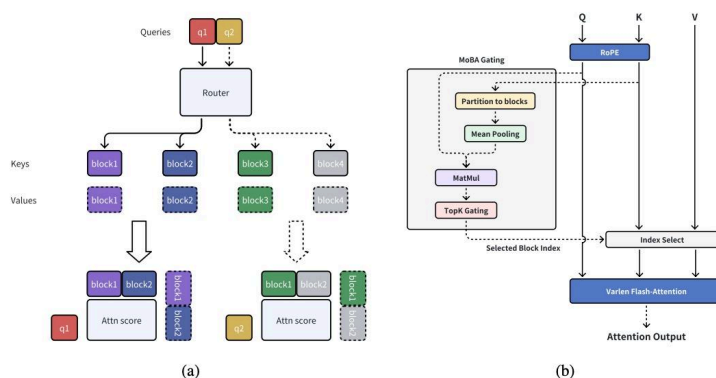


Figure 1: Illustration of mixture of block attention (MoBA). (a) A running example of MoBA. (b) Integration of MoBA into Flash Attention.

其中 MoBA 关键的技术如下：

- 细粒度上下文分割 (Fine-grained Contextual Blocking) 与稀疏注意力路由 (Sparse Routing)
1. 类似于 MoE 中的专家路由机制，MoBA 将输入序列按上下文维度而非 FFN 维度划分为多个小块 (blocks)。
 2. 每个 block 专注于部分上下文，避免全局计算，从而降低计算量。
 3. 类似 MoE 中的 token-专家选择机制，MoBA 为不同的块选择特定的注意力头进行处理，避免冗余计算。

$\text{Attn}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}(\mathbf{q}\mathbf{K}^\top) \mathbf{V}$, 其中 $\mathbf{q} \in \mathbb{R}^{1 \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 。MoBA改进了这一机制，查询只关注选定的块：

$$\text{MoBA}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}(\mathbf{q}\mathbf{K}_{[I]}^\top) \mathbf{V}_{[I]},$$

其中 $I \subseteq [N]$ 是选定键值对的索引集，由门控机制决定：

$I = \bigcup_{g_i > 0} I_i$, $I_i = [(i-1) \times B + 1, i \times B]$. 门控值 g_i 通过查询与块的亲和度得分 s_i 计算：

$$s_i = \mathbf{q}^\top \cdot \text{mean_pool}(\mathbf{K}_{[I_i]}),$$

$$g_i = \begin{cases} 1 & s_i \in \text{Topk}(\{s_j | j \in [n]\}, k) \\ 0 & \text{otherwise} \end{cases}.$$

知乎 @Zachary

• MoBA 与 Full Attention 混合训练

1. 支持 初始化阶段动态选择使用 MoBA 或 Full Attention。
2. 支持训练过程中逐步过渡或融合，从 Full Attention 迁移到 MoBA，保证收敛性和性能

如下图所示，左图在 1M 序列长度计算效率提升评估中，随着序列长度从 8K 增加到 1M，MoBA 的计算时间扩展显著优于全注意力（使用 Flash Attention 实现）；右图在固定稀疏比率的计算时间扩展对比中，序列长度从 8K 增加到 10M，保持 95.31% 的稀疏率（固定 64 个 MoBA 块，块大小可变，top-k=3），MoBA 的效率优势进一步凸显：

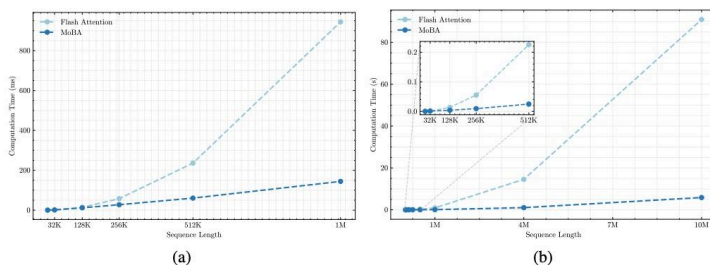


Figure 2: Efficiency of MoBA vs. full attention (implemented with Flash Attention). (a) 1M Model speedup evaluation: Computation time scaling of MoBA versus Flash Attention on 1M model with increasing sequence lengths (8K-1M). (b) Fixed Sparsity Ratio scaling: Computation time scaling comparison between MoBA and Flash Attention across increasing sequence lengths (8K-10M), maintaining a constant sparsity ratio of 95.31% (fixed 64 MoBA blocks with variance block size and fixed top-k=3).

Native Sparse Attention

核心思想：寻找最适合的稀疏方式。NSA 首先将上下文进行分块，随后分别放在不同的稀疏处理机制中，并采用门控机制选择最有效的注意力结果。其中门控单元是可学习的。

数学公式：

其中 $\mathbf{K}_t, \mathbf{V}_t$ 是动态更新的键值对, $\mathbf{K}_t = JK(\mathbf{Q}_t, \mathbf{K}_{:t}, \mathbf{V}_{:t})$, $\mathbf{V}_t = JV(\mathbf{Q}_t, \mathbf{K}_{:t}, \mathbf{V}_{:t})$.

最终输出是三条路径的加权和:

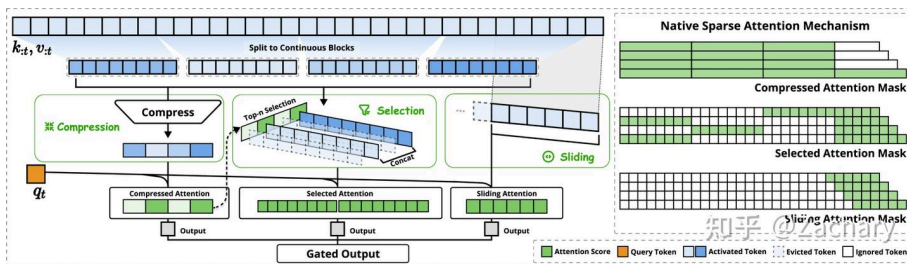
$$\mathbf{o}_t^* = \sum_{c \in \{\text{cmp, slc, win}\}} g_t^c \cdot \text{Attn}(\mathbf{q}_t, \tilde{\mathbf{K}}_t^c, \tilde{\mathbf{V}}_t^c),$$

其中 $g_t^c \in [0, 1]$ 是通过MLP和sigmoid激活计算的门控分数。

- **压缩路径**: 将键值对聚合成块级表示: $\tilde{\mathbf{K}}_t^{\text{cmp}} = \left\{ \phi(\mathbf{k}_{id+1:id+l}) \mid 0 \leq i \leq \left\lfloor \frac{t-l}{d} \right\rfloor \right\}$, 其中 ϕ 是一个带位置编码的MLP, l 是块长度, d 是滑动步幅。
- **选择路径**: 根据查询选择关键token: $\tilde{\mathbf{K}}_t^{\text{slc}} = \{\mathbf{k}_i \mid i \in \mathcal{I}_t\}$, $\mathcal{I}_t = \psi(\mathbf{q}_t, \mathbf{k}_{:t})$, ψ 是一个两层MLP, 输出每个token的选择得分, 取Top-k。
- **滑动窗口路径**: 捕获局部上下文: $\tilde{\mathbf{K}}_t^{\text{win}} = \{\mathbf{k}_{t-w:t+w}\}$, 其中 w 是窗口大小。

核心设计 (三种稀疏策略):

1. Token Compression: 通过将序列进行分块, 并用可学习的MLP将整个块的KV映射到单个KV。
2. Token Selection: 使用压缩后的KV计算每个块的重要性, 并选取top-n个块进行计算。
3. Sliding Window: 只选用最近的局部KV进行注意力的计算。



我们之前做过在LLAMA上三种稀疏策略的效果 (推理侧), 实际上Token Selection的方法最有用, 即所用层无脑用分块策略即可, 但不知道NSA实际上训练的效果如何。

优点: 基于一种可学习的门控单元对多种稀疏策略进行仲裁, 选择最有效的稀疏结果, 能够根据不同长上下文场景遇到的不同类型问题进行针对性处理。

MIInference

MIInference使用的方法, 相当于是streamllm、h2o与snapkv的结合体, 当prompt长度为128K时, 若在attention中只取Top-4K的列进行运算, 则可以召回超过96%的全局attention score。

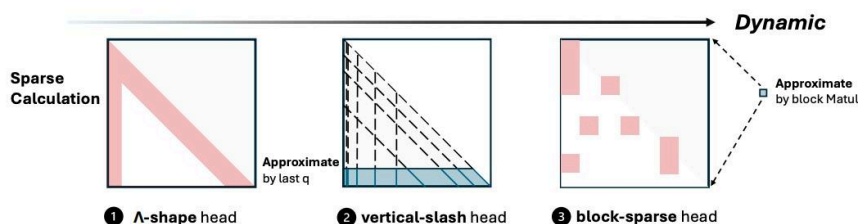


Figure 4: The three sparse methods in MIInference.

当中有点不合理的地方, 是块的选择用池化后的QK分块直接相乘, 如下图所示:

$A_b[i, k]$ 表示第 i 个 query block 对第 k 个 key block 的估计注意力强度。

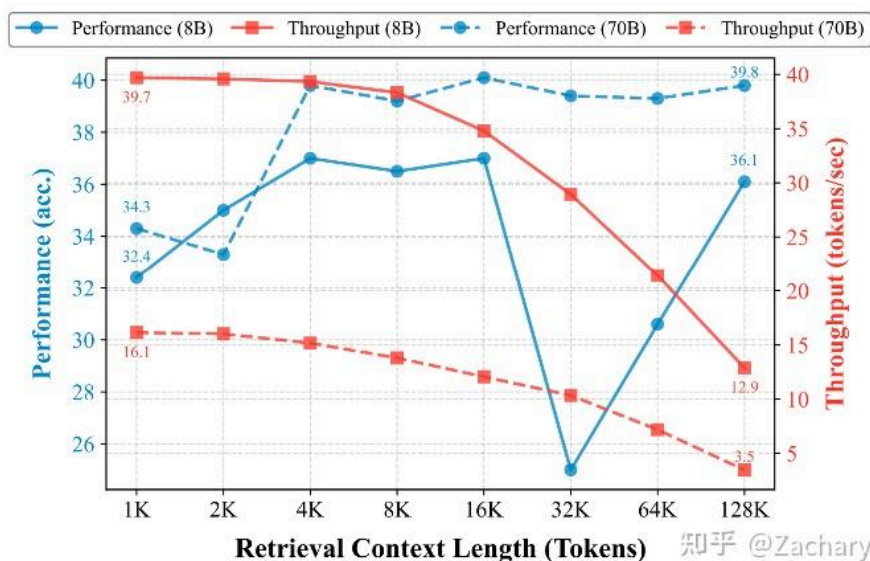
我这块认为 block-sparse 可能会更有用，其余两种稀疏并没有必要，并且A的打分应该为 softmax 的打分会更合理一点。

Speculative Decoding

在处理百万级 token 的长上下文时，传统 speculative decoding 的性能优势基本失效。这是因为 KV Cache 的大规模加载与验证成为主导瓶颈，并且 draft 模型维护自己的 KV Cache 亦需巨量显存。此外，长上下文与 draft 模型训练分布不一致导致的 acceptance rate 下降，也进一步削弱 speculative 带来的加速效果。因此，为了在长上下文场景中继续提速，必须结合 KV cache 压缩、固定窗口草稿模型、检索增强推理等机制，以实现实用的性能提升路径。

RAPID

虽然传统的 *Speculative Decoding* (SD) 在标准长度上下文下可以显著加速，但在处理超长文档时，由于内存受限的 KV 缓存操作，其优势会大大减弱。



RAPID 所依赖的理论依据，如上图所示，用 RAG 起草器时，长度为 4k 的 context length 在对应的精度上等同于 128k 长度的上下文，因而可以先通过 RAG 对 context 进行过滤，然后再输入到草稿模型生成候选集，最终通过目标模型过滤。

RAPID 包含两个关键组件：

- RAG 起草器 (RAG Drafter)

RAPID 使用轻量级的 RAG 起草器 代替传统的草稿模型。在长上下文环境中，传统 SD 会因需要同时在小模型和目标大模型中保留完整上下文的 KV Cache，从而导致资源开销巨大、加速无效。

RAG 起草器通过 检索相关上下文片段 来生成候选输出，只访问与当前问题高度相关的少量上下文窗口。这样不仅显著减少了内存负担

在长上下文下的推理加速，具体的数学公式：

• q_θ : RAG drafter 的 token 分布。

• C^S : 通过检索选出来的上下文子集, 满足 $|C^S| \ll |C|$ 。

• 选取 C^S 的方式是将 C 分成若干段落, 编码成 dense 向量, 然后根据语义相似度 (如 cosine similarity) 来选取与当前 query 最相关的若干段落。

检索增强的目标分布 (Retrieval-Augmented Target Distribution)

传统 SD 的验收机制依赖于目标模型严格匹配草稿模型的输出概率分布, 限制较大, 尤其在 RAG 生成的候选较强时, 往往会误拒优质答案。

RAPID 通过引入 **检索增强的目标分布**, 在目标模型的评估过程中, 允许它参考 RAG Drafter 的输出作为「先验信息」, 从而实现更智能的验收策略, 因而对接受率公式进行如下调整, 该机制在保留原始 SD 理论保证的同时, 增强了验收率, 提升了最终的推理效率和质量。

$$r \leq \min \left(1, \frac{p(x_i)}{q(x_i)} \right) = \min \left(1, \frac{p_\phi(x_i | [C; x_{<i}])}{q_\psi(x_i | [C^S; x_{<i}])} \right)$$

我当前也在基于 BGE-M3/Qwen2 embedding 在跑一下基于 deepseek 的精度测试, 看看在 prompt 层做过滤的效果。

SuffixDecoding⁺

SuffixDecoding 采用另外一种思路解决草稿的性能问题,

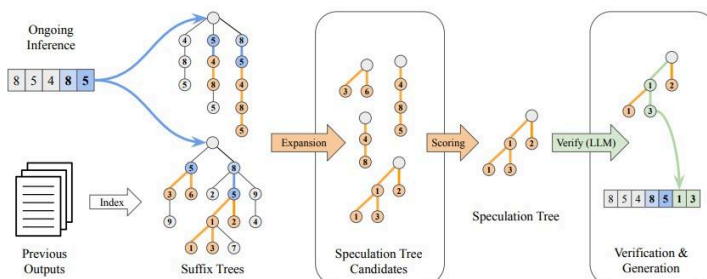


Figure 1. Overview of SuffixDecoding's workflow. The system maintains two suffix trees: one built from the ongoing inference (shown on top-left) and another from previous outputs (shown on bottom-left). When generating tokens, SuffixDecoding uses these trees to find matching patterns based on recently generated tokens. It then constructs a speculation tree (middle-right) by selecting the most likely continuations from the suffix trees, scoring them based on frequency statistics. The selected candidates are arranged in a tree structure to allow parallel verification. Finally, these candidate tokens are verified by the LLM in a single forward pass (right), with accepted tokens (shown in grey) being added to the output and used for the next round of speculation. This process enables efficient speculation without requiring additional models or specialized heads.

上图为 SuffixDecoding 整体流程, 主要分以下步骤:

- (1) 首先构建并维护两个后缀树 (suffix trees), 一个是来自推理过程产生的, 一个来的语料中;
- (2) 接着, 根据解码产生的 tokens, 从后缀树生成候选的采样后缀树, 并利用统计指标进行打分排序;

$$C(N) = \frac{\text{COUNT}(N)}{\sum_{M \in \text{CHILDREN}(\text{PARENT}(N))} \text{COUNT}(M)}$$

$$D(N) = \begin{cases} D(\text{PARENT}(N)) \times C(N), & \text{if } N \neq N_p \\ 1, & \text{otherwise} \end{cases}$$

为匹配 token 的后续 token 的可能性大小。

(3) 最后利用 LLM 对 top-k 的候选后缀树序列进行并行验证，得到模型接受的序列作为解码结果，即将分数较高的候选子树输入到 LLM 进行验证和生成；最后递归上述过程，直至满足生成结束条件。

Dataset	Throughput (tokens/sec)				Per-request TPOT (ms)				Avg. Step Time (ms)			
	SD-T	SD-L	SPEC	INC	SD-T	SD-L	SPEC	INC	SD-T	SD-L	SPEC	INC
AgenticSQL (Enrich)	354	251	139	134	18.4	28	51.3	55.9	110	79.5	254	56.8
AgenticSQL (Classify)	273	278	160	129	25	25.4	44.5	57.7	99	90	256	58.1
AgenticSQL (Extract)	632	544	217	142	9.99	11.3	29.8	49.2	164	131	261	49.9
AgenticSQL (SQL1)	219	216	155	123	31.3	30.7	46.5	59.8	96.1	90	321	60.4
AgenticSQL (SQL2)	347	324	278	151	20.3	21.4	26.9	49	53.8	53.7	171	49.2
AgenticSQL (SQL3)	216	218	155	118	30.6	26.1	45.8	61	99	78.3	317	61.6
AgenticSQL (Combine)	424	387	197	132	15.8	20	35.5	56.1	97.4	106	279	56.6
Spider	183	132	127	112	36	36.1	41.3	64.8	108	88.6	355	68.2
WildChat	350	238	281	163	30.4	35.8	28.4	48.3	63	58.5	160	48.5
MagiCoder	424	299	328	164	25.2	28.8	24	48.2	57.4	55.8	156	48.3

Table 1. Throughput, TPOT, and Average Step Time for Different Decoding Methods. SuffixDecoding with tree speculation (SD-T) is the default SuffixDecoding algorithm, selecting a subtree of candidate tokens from the suffix tree, and passing it to the LLM for verification. Instead, SuffixDecoding with linear speculation (SD-L) only allows a linear chain of candidate tokens to be selected from the suffix tree. Speculative Inference (SPEC) is a baseline tree-based speculative decoding baseline (Miao et al., 2024), which uses a draft model with top-k sampling to generate trees of candidate tokens. Incremental Decoding (INC) is a traditional LLM inference baseline that does not employ speculative decoding and generates one token per decoding step.

其中 SD-T 即为论文提出的 SuffixDecoding 解码策略，其他为变体和其他解码策略。显示 SD-T 在每秒解码速度是最快的，比 INC(传统解码方式，如 topk, greedy 等)解码方式快近 3 倍左右。此外，在 TPOT（每个 token 延迟时间）上，SD-T 在大部分数据集是最小的。

Speculative Prefill

Speculative Prefill 想解决的是长文本 prefill 过慢的问题，也就是降低 time-to-first-token (TTFT)。方法很自然：使用一个轻量级的“小模型”（称为 Speculator）来预测哪些 token 是重要的，然后只把这些 token 和对应的位置信息传给大模型（主模型）去处理。具体操作流程如下：

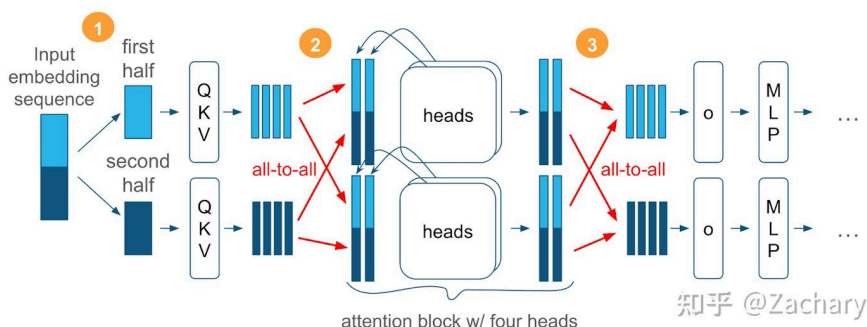
1. 先用一个小模型看一遍 prompt，计算 attention 分数。使用最后一个 token 或 Look-ahead（往后预测几个 token）的 attention 分布来评估每个 token 的重要性。这里使用 Look-ahead 是因为最后一个 token 的 attention score 可能会过度关注离自己较近的 token，所以需要拉远一点距离。
2. 通过 max-mean 这种简单的聚合策略，得到每个 token 的重要性得分。分 chunk（比如每 8 个 token 为一组）平均这些得分，并挑出最重要的几个 chunk。
3. 被选中的 token 要保持它们原来的位置编码（Position IDs），以便主模型能正确处理它们。比如原本有 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]，可能只保留了 [0, 2, 4, 7]，那么传回大模型的时候，如果新生成了三个 token，Position ID 应当是 [0, 2, 4, 7, 10, 11, 12]。这样才能保持这个稀疏的位置信息。

在 QA 这种任务上，即使只保留 10% 的 prompt 也没有掉很多点。但在 Summarization 和 CodeGen 就需要保留比较多的 prompt。这其实是能够理解的。毕竟在 query 已知的情况下，这更像是一种 RAG。

Sequence Parallelism

Arctic Ulysses⁺

在注意力计算中，虽然序列内的 token 存在依赖关系，但不同注意力头之间可以并行计算。Ulysses 利用这一特性，通过两次 all-to-all 通信，在计算注意力前将并行策略从序列并行（SP）切换为注意力头并行。这样，每个 GPU 从原本持有全部注意力头的部分序列，变为持有完整序列中部分注意力头，从而并行完成各自负责的注意力计算。完成后再通过 all-to-all 通信切回序列并行布局，实现高效计算与



通信与计算比率分析

Arctic Ulysses 有效地消除了具有固定通信量的 all-reduce 通信,取而代之的是两次通信量随 SP 度减少的 all-to-all 通信。如表 2 所示,SP 在减少每个 GPU 的通信和计算的同时保持它们的比率恒定。

	Per-GPU Complexity			
	Memory	Compute	Comm. Volume	Comm./Compute
TP	$m(n,w)/TP$	$f(n,w)/TP$	$c(n,w)$	$TP \times \text{const}$
SP	$m(n,w)$	$f(n,w)/SP$	$c(n,w)/SP$	const
spxtp	$m(n,w)/tp$	$f(n,w)/(tp \times sp)$	$c(n,w)/sp$	$tp \times \text{const}$

表 2. TP、SP 及其组合(表示为 $sp \times tp$)的计算复杂度。这里, tp 表示比 TP 更小的张量并行度。为了兼顾内存效率与可扩展性, Arctic Ulysses 将序列并行 (SP) 与张量并行 (TP) 结合使用。TP 通过在多个 GPU 之间切分模型权重,使超大规模模型能够适应聚合内存,而 SP 在此基础上叠加,利用其高效的通信与计算特性进一步提升扩展性并降低首次 token 推理时间 (TTFT)。这种混合并行方式 ($tp \times sp$) 不仅提升了计算效率,也显著优化了内存使用,非常适用于大规模、长上下文、低延迟的推理场景。

ZeCO⁺

ZeCO (Zero Communication Overhead Sequence Parallelism) 主要解决了线性 Attention 在超长序列训练中面临的**高通信开销瓶颈**问题。尽管线性 Attention 具有仅需累加状态的优势,传统的序列并行方法仍需频繁地在设备间同步这些状态,从而产生大量通信负担。ZeCO 的关键技术是引入 **All-Scan**, 一种专门为线性 Attention 设计的高效前缀传播原语,它允许每个 GPU 通过一次轻量的集体通信快速获取所需的累加前缀状态,彻底避免逐步同步带来的延迟和带宽占用,从而实现端到端的**近线性可扩展性**。实验证明 ZeCO 能在 256 卡上训练 8M 长度序列,比现有方法快 60%。尽管 ZeCO 的优势主要体现在训练阶段,但其设计原理(减少通信、前缀累积优化)也对**分布式推理中处理超长上下文**具有启发意义,尤其适用于未来更高吞吐、低延迟的大模型系统。

长上下文 10 倍加速的思考与实践

在当前的实际工业应用之中,尤其是 Deepseek V3 等模型,长上下推理并未真正的全面落地,甚至部分 API 服务商以滑窗截取的方式提供服务

- Linear Attention, Linear Attention 虽然可以将计算复杂度从二次降低为线性, 但模型的精度会受到一定影响, 同时 Hybrid 方式的模型结构仍旧存在 softmax Attention, 依然会面临计算复杂度上的瓶颈问题。
- Sparse Attention, 训练侧的 MoBA 与 NSA 的方案, 目前 NSA 尚未有成熟的模型进行发布, MoBA 的成熟度和稳定性可能还需要时间来验证, 基于动态稀疏注意机制的算法大多是理论上的逻辑, 并未真正的落地。
- Speculative Decoding, 虽然有 RAG 或者 SuffixDecoding 可以避开草稿本身的性能问题, 但目标模型的计算代价以及高并发时的吞吐性能, 并未真正的解决。
- Sequence Parallelism, 相对解决 TP 带来的通信问题, 功能上可以支持长上下文, 但最关键的计算复杂度并未真正的解决。

综上, 各个方法各有优劣, 还有很多其它思路非常好的论文没有列举, 总体上都是理论非常美好, 但在工程落地会上非常困难。

有没有一种方法, 能十分妥当的在工程上落地 (最好能以 plugin 的方式注入进去), 同时兼容主流的 vLLM/sglang, 支持 dynamo/mooncake 等 PD 分离, 与开源的 deepseek 方案复现不互斥, 同时实现十倍的推理加速并且精度基本不掉点, 同时还支持 Deepseek V3、LLAMA 等多种模型?

OK, 关于上述比较全面的问题, 我的思路是通过动态稀疏注意机制实现上述的全部诉求, 尤其是 Deepseek 等大型 MOE 模型的分布式推理, 首先我们附上一张苏神关于 MLA 的数学公式总结 (具体公式参考: spaces.ac.cn/archives/1...) :

$$\mathbf{o}_t = [\mathbf{o}_t^{(1)}, \mathbf{o}_t^{(2)}, \dots, \mathbf{o}_t^{(h)}]$$

$$\mathbf{o}_t^{(s)} = \text{Attention}(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(s)}, \mathbf{v}_{\leq t}^{(s)}) \triangleq \frac{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top}) \mathbf{v}_i^{(s)}}{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top})}$$

$$\mathbf{q}_i^{(s)} = [\mathbf{c}_i' \mathbf{W}_{qc}^{(s)}, \mathbf{c}_i' \mathbf{W}_{qr}^{(s)} \mathcal{R}_i] \in \mathbb{R}^{d_k + d_r}, \quad \mathbf{W}_{qc}^{(s)} \in \mathbb{R}^{d_c' \times d_k}, \mathbf{W}_{qr}^{(s)} \in \mathbb{R}^{d_c' \times d_r}$$

$$\mathbf{k}_i^{(s)} = [\mathbf{c}_i \mathbf{W}_{kc}^{(s)}, \mathbf{x}_i \mathbf{W}_{kr}^{(s)} \mathcal{R}_i] \in \mathbb{R}^{d_k + d_r}, \quad \mathbf{W}_{kc}^{(s)} \in \mathbb{R}^{d_c \times d_k}, \mathbf{W}_{kr}^{(s)} \in \mathbb{R}^{d_c \times d_r}$$

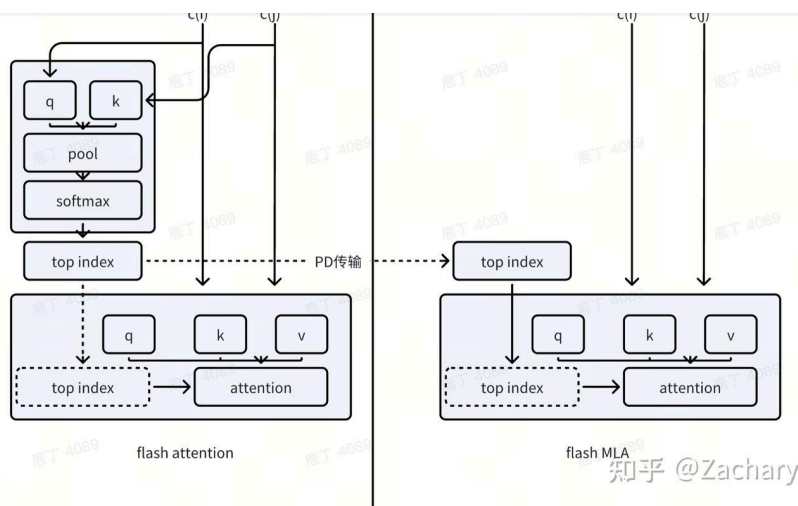
$$\mathbf{v}_i^{(s)} = \mathbf{c}_i \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_v}, \quad \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_c \times d_v}$$

$$\mathbf{c}_i' = \mathbf{x}_i \mathbf{W}_c' \in \mathbb{R}^{d_c'}, \quad \mathbf{W}_c' \in \mathbb{R}^{d \times d_c'}$$

$$\mathbf{c}_i = \mathbf{x}_i \mathbf{W}_c \in \mathbb{R}^{d_c}, \quad \mathbf{W}_c \in \mathbb{R}^{d \times d_c} \quad \text{知乎 @Zachary}$$

从上述的公式中, 我们可以得知在 Attention 部分的计算逻辑, MLA 与 MHA\MQA\GQA 并无差异, 甚至说 MLA 是一种在 MHA 基础上添加低秩投影和只在部分维度加 RoPE 外的变种, 因而 MLA 同样支持动态稀疏注意机制。

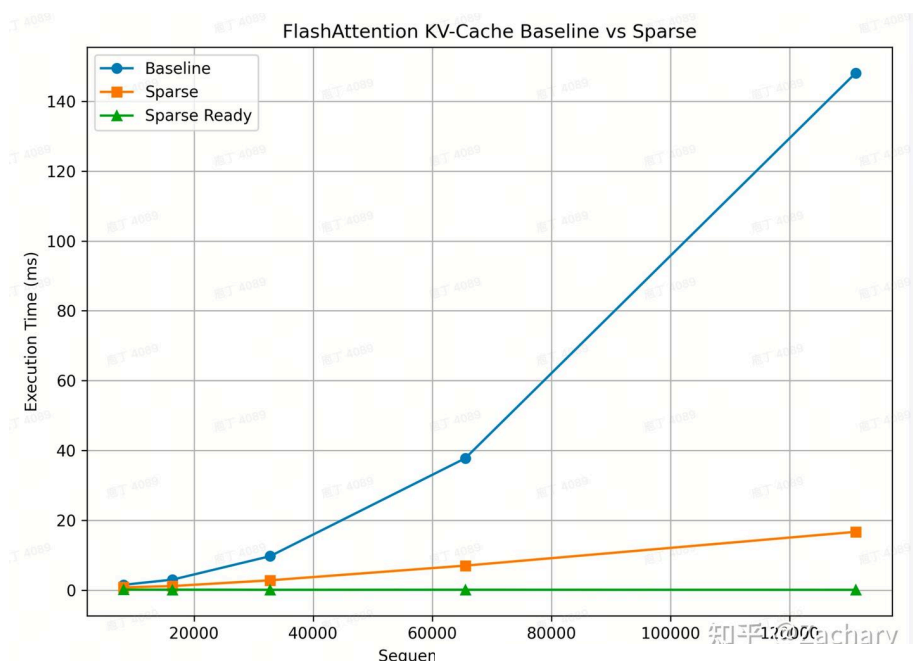
基于以上的结论, 我们简答的画一张草图, 说明怎么去实现 Deepseek 的动态稀疏注意机制, 如下图所示:



简而言之，上述的草图主要分成以下的步骤：

- Block index计算，即在进行attention计算之前，先通过q和k去计算出各个head上不同block的重要性（block可以用kv cache的block id平替），具体的算法用softmax算法（虽说softmax是pass 2的算法，但我们实际测试过，在flash attention基础上适当修改，性能损失几乎只有3%左右），然后选出topk。此步骤因为pool的缘故，性能不是瓶颈。
- Prefill的attention计算，正常的flash attention计算即可，需要把上一步的top index加入到flash attention中，底层kernel寻址时，替换block table，用top index进行寻址，这种计算量就会大大的降低，理论上，可以将128k的上下文给压缩到4k而精度几乎无损。具体的性能数据与算子实现，可以参考我之前的文章：[通过KV稀疏对vLLM实现1.5倍的推理加速](#)
- top index传输，即将top index从Prefill传输到Decode，即便top index多了一个head_num的维度，但实际上只传输block_id，再加上block压缩的缘故，此步骤的代价可以忽略不计。
- Decode的attention计算，与Prefill类似，加入top index进行寻址，进一步降低计算量与访存开销。

OK，上面是工程架构或者流程上的一些设计，最核心的还是要看真正的性能效果，我们基于flash-atten中的flash_attn_with_kvcache函数，给出当前的测试结果，如下图所示：



疏attention函数，sparse ready是稀疏attention函数所依赖的前置计算，从上图明显可以看到，当序列的长度为128k时，几乎有着10倍的加速。

总体上，我认为基于动态稀疏注意机制的方法完全成立，完全可以去解决Deepseek等大型MOE模型的长上下文推理问题，尤其是在工程实现上是完全可行的，无论是计算复杂度、带宽瓶颈，还是系统的兼容性问题，暂时没有看到比较明确的风险。补充一点的是，基于attention的动态稀疏注意机制，与MOE的门控稀疏机制，是一个完全正交的逻辑，有可能在长上下文和大模型推理中，实现 token-level 动态稀疏推理。

参考文献

TPTT: Transforming Pretrained Transformer into Titans
MoBA: Mixture of Block Attention for Long-Context LLMs
Lightning Attention-2: A Free Lunch for Handling Unlimited Sequence Lengths in Large Language Models
Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention
MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention
RAPID: Long-Context Inference with Retrieval-Augmented Speculative Decoding
SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications
Retrieval-Augmented Generation for Large Language Models: A Survey
cAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Tree
CodeGRAG: Bridging the Gap between Natural Language and Programming Language via Graphical Retrieval Augmented Generation
Speculative Prefill: Turbocharging TTFT with Lightweight and Training-Free Token Importance Estimation
Low-Latency and High-Throughput Inference for Long Context with Sequence Parallelism (aka Arctic Ulysses)
ZeCO: Zero Communication Overhead Sequence Parallelism for Linear Attention

编辑于 2025-08-22 10:48 · 北京

推理加速 长上下文 DeepSeek

个性化学习从这开始：手把手教你建题库

在备考或技能提升的道路上，拥有一个适配个人需求的刷题工具，往往能让学习效率倍增。今天向大家推荐一个实用方法——借助Bangboss在线考试系统，你可以轻松构建属于自己的专属题库...

Bangboss 100+热议



理性发言，友善互动

1 条评论

默认 最新



夜.月

『基于attention的动态稀疏注意机制，与MOE的门控稀疏机制，是一个完全正交的逻辑，有可能在长上下文和大模型推理中这怎么理解？动态稀疏注意力+MoE不

推荐阅读

OnePiece：上下文工程与隐式推理，重构模型思考能力

前言 2025 年，生成式推荐（Generative Recommender，GR）的发展如火如荼，其背后主要的驱动力源自大语言模型（LLM）那诱人的 scaling law 和通用建模能力（general-purpose...
傅聪 Cong

从零实现 AI 推理引擎 04 JIT 编译

在这部分里，我们会为之前定义的算子集实现 CUDA 后端（写一堆 kernel），但是这一章里我们暂时不打算实现任何的 CUDA kernel，而是先做一些必要的准备工作。这里假设读者对 Linux 上的 CUDA 与...
闭门造车

Light-DuoAttention：用 CuTeDSL 实现高效长上下文...

引言：长上下文的挑战想象一下，你正在使用一个大语言模型处理一份 100 页的技术文档，然后在文档的某个角落问它一个具体问题——这就是著名的“大海捞针”（Needle in a Haystack,...
KuangjuX

和推理引擎对抗的日子

自年底完成[1]后，算是阶段性对 long-cot 蒸馏（包括多模态）有了基础实践和认知后，就进入第二阶段，zero-rl。彼时，正好有 verl 和 openrlhf 两个框架，笔者更熟悉 openrlhf 的写法，便开始了 open...
haotian